CS G525 ADVANCED COMPUTER NETWORKS · BITS PILANI

Progress Review II

Project SA7 · Reactive vs. Proactive Flow Installation in OpenFlow SDN

Project: Empirical Evaluation of Reactive vs. Proactive Flow Installation in OpenFlow SDN

Group: 2

Members: N. NISARG VAGH, D. DHADUK MRUDUL HITESHBHAI, PRACHIT SURESH DESHINGE, Mukul Saini

Review date: 30 October 2026 (Progress Review II: working implementation, full experiments, preliminary analysis)

Code and raw data: https://github.com/PrachitDeshinge-324/Openflow_SDN

---

0. At a glance

The question. In OpenFlow SDN, a switch needs a rule before it can forward a packet. There are two ways to get one:

- **reactive:** ask the controller when the first packet of a flow arrives;
- **proactive:** install every rule in advance.

Which is better, by how much, and when?

Everything below is measured on Mininet 2.3.0 + Open vSwitch 3.7.1 (kernel datapath) + POX 0.7.0. It ran on our laptops (Windows 11, WSL2) with our own instrumented controllers.

Since Review I we added	Headline result (reactive vs proactive)
Networks with many paths: a data-centre **fat-tree** and the real **Abilene** ISP backbone	Reactive's paths are up to **4.5× longer in delay**. Real pings are **51% slower** (40.5 vs 26.9 ms). Proactive always uses a best path.
Load balancing (ECMP) as a third controller	In the fat-tree, ECMP carries **50% more traffic** than reactive (95 vs 64 Mbit/s) and leaves only 16% of links idle (reactive leaves 59%).
University traffic (Benson et al., IMC 2010)	Small flows: **median 55.7 ms reactive vs 1.3 ms proactive (44×)**. The controller ran at **86% CPU** with 2,379 requests/s.
Data-centre traffic (DCTCP web search, VL2 data mining)	Small flows are **8–10× faster with proactive ECMP**, and large flows **3–3.6× faster**.
Real ISP traffic (Abilene traffic matrices, 2004)	Small flows: **median 120 vs 48 ms (2.5×)**, of which ~27 ms is longer paths and ~40 ms is controller set-up.
Review I **reproduced on a second laptop**	Same patterns, identical rule counts.

Every flow finished in every run (0 failures), and every controller got exactly the same traffic.

---

1. Response to the Review I feedback

Review I score: 7.5 / 10. The feedback, and what we did:

Feedback	What we did	Where
"Previous eval done and improvements done"	This report: Review I recap (Section 2), all new work (Sections 4–6), reproduced on a second laptop	Sections 2, 4, 5
"Increase the traffic like uni traffic, data centre, ISP"	Four realistic workloads, each from a published measurement: university, data-centre web search, data-centre data mining, and real Abilene ISP traffic matrices	Section 5
"Parameters for evaluation: optimal path vs haphazard path"	New networks with many paths. We measured path stretch, delay, throughput and link balance for reactive (haphazard), proactive (optimal) and proactive ECMP	Section 4
"Cmp with previous works / papers which already has predictor, traffic gen, emulator, etc."	Literature review of 14 papers (11 checked in full text). Our numbers are set next to theirs, and every network and traffic choice is traced to a paper. Re-implementing their methods as baselines is planned for the final submission.	Section 6, `docs/literature/`, `docs/Realism_Sources.md`
"Run on multiple comps using distributed switches in lab"	Not yet. Planned next: controller on one PC, Mininet on another (real control-channel delay), then Mininet cluster mode across lab PCs	Section 8

---

2. Recap: Review I results, now reproduced

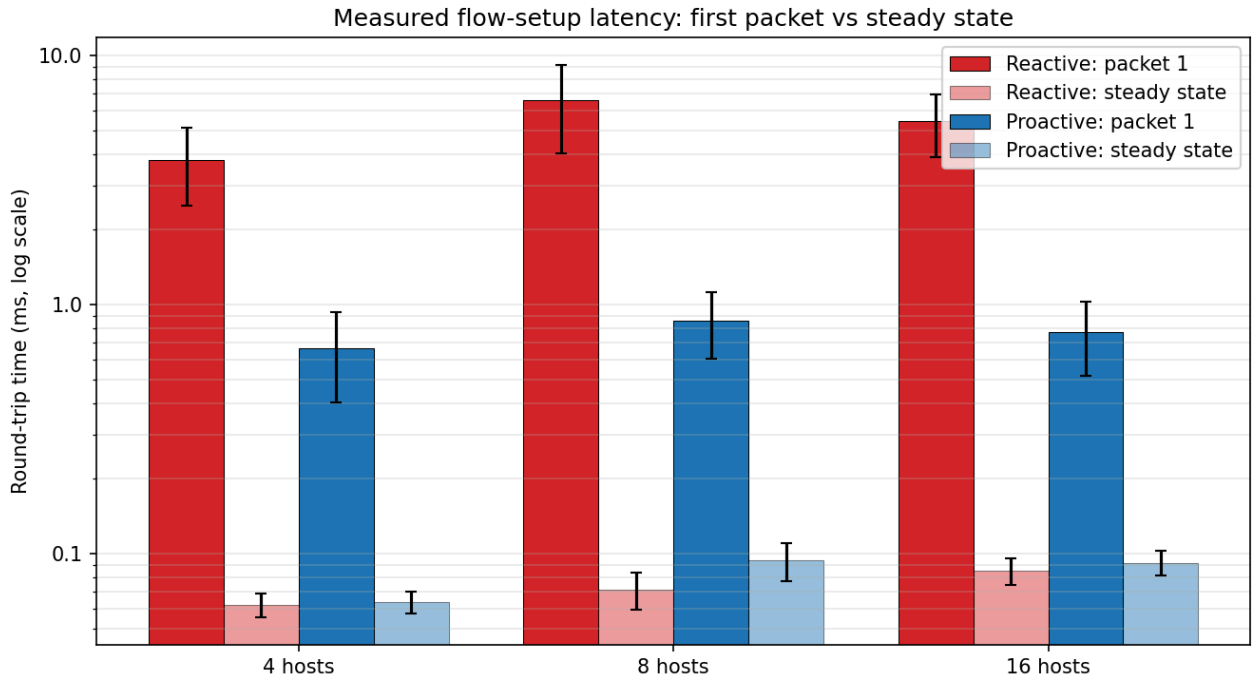
Setup. Tree networks with 4, 8 and 16 computers. Two instrumented POX controllers:

- `reactive_eval`: the same logic as POX's `l2_learning`, with exact-match rules (idle 10 s, hard 30 s);
- `proactive_eval`: one rule per host pair on its shortest path, installed when the switch connects.

Measurement (laptop 1, 27 Sep)	4 computers	8 computers	16 computers
First packet, reactive (ms)	3.81	6.60	5.44
First packet, proactive (ms)	0.67	0.86	0.77
Later packets, both (ms)	0.06–0.09	0.07–0.09	0.09
Rules, proactive / reactive with all pairs active	28 / 28	216 / 216	624 / 624
Rules after 5 s of 200 new flows/s: reactive vs proactive	2,321 vs 28	3,857 vs 216	2,631 vs 624
Controller at 200 new flows/s, reactive: PACKET_IN/s, CPU	465, 20.8%	786, 36.4%	544, 31.6%

Reproduced on laptop 2 (9 Oct). Same software, a different machine:

- First packet, reactive: 7.2 / 11.4 / 8.7 ms; proactive: 1.0 / 1.5 / 1.0 ms.
- Rule counts were **identical**.
- At 200 new flows/s: 477 / 782 / 535 PACKET_IN/s.
- The 8-computer tree was again the slowest. It has the longest paths (4 switches), and reactive sets rules up switch by switch.



Review I latency

Review I findings in short.

1. Reactive's first packet is 5–8× slower, and the cost grows with the number of switches on the path.
2. Later packets are equal.
3. When every pair talks, proactive uses no more rules than reactive. Under churn, reactive's exact-match rules pile up to 4–83× more.

3. Testbed and method (common to all experiments)

Item	Value
Emulator / switch / controller	Mininet 2.3.0, Open vSwitch 3.7.1 (kernel datapath), POX 0.7.0 (`gar`), OpenFlow 1.0
Machine	Laptops with Windows 11 + WSL2 (Ubuntu, kernel 6.18), Python 3.14
Controllers	**Reactive:** learning switch, exact-match rules, idle 10 s / hard 30 s. In networks with loops it runs over POX `openflow.discovery` + `openflow.spanning_tree`. **Proactive:** one rule per host pair on the lowest-cost path, plus a table-miss drop rule. **Proactive ECMP:** host pairs hashed (CRC32 of the MACs) across equal-cost paths.
Same conditions	Static ARP, IPv6 off, fixed random seed. **Every controller gets exactly the same pairs, traffic permutations and flow schedules.**
Measured, not modelled	Paths are read from the switches' flow tables (`ovs-ofctl dump-flows`); latency with ping / TCP flow completion times; controller load from counters inside our controllers; link loads from interface byte counters
Realism	Every network, traffic pattern and parameter is traced to a paper in `docs/Realism_Sources.md` (16 references)

4. Experiment A: optimal path vs haphazard path

Why. A tree has one path between two hosts, so all controllers used the same path in Review I. Real networks have many paths.

Networks.

- **fat-tree, k = 4** (data centre; Al-Fares, SIGCOMM 2008): 20 switches, 16 hosts, up to 4 equal paths between pods.
- **Abilene** (real US research backbone; Internet Topology Zoo): 11 cities, 14 links. Each link delay is the city distance ÷ 200 km/ms (light in fibre), emulated with netem.
- **Link speed:** 10 Mbit/s on every link, so only the path choice matters.

Controllers.

- Reactive = learning switch over a spanning tree. Floods follow the tree, so paths are whatever the tree gives: **haphazard**.
- Proactive = lowest-delay path: **optimal**.
- Proactive ECMP = optimal paths, spread over the equal-cost ones.

Traffic.

- **A (paths):** 40 host pairs, 6 pings each. The real path is traced switch by switch.

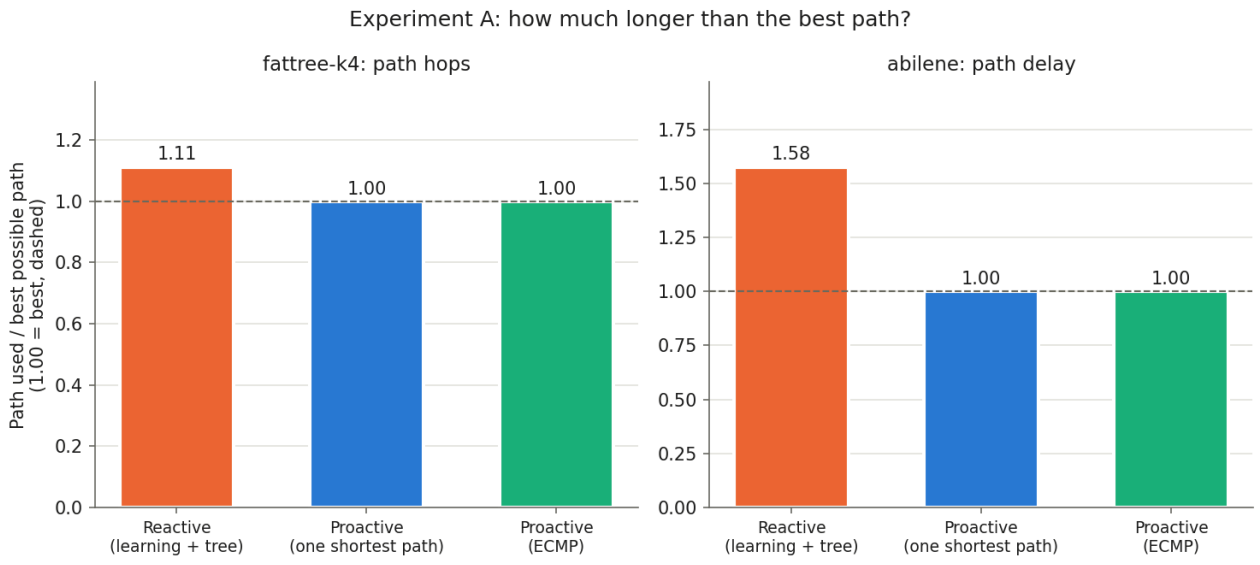
- **B (throughput):** every host sends one TCP flow to another at the same time (permutation traffic, as in Al-Fares 2008 / Hedera 2010), 3 permutations × 10 s.

Results (laptop, 15 Oct; `results/measured/paths-20261015-094800/`)

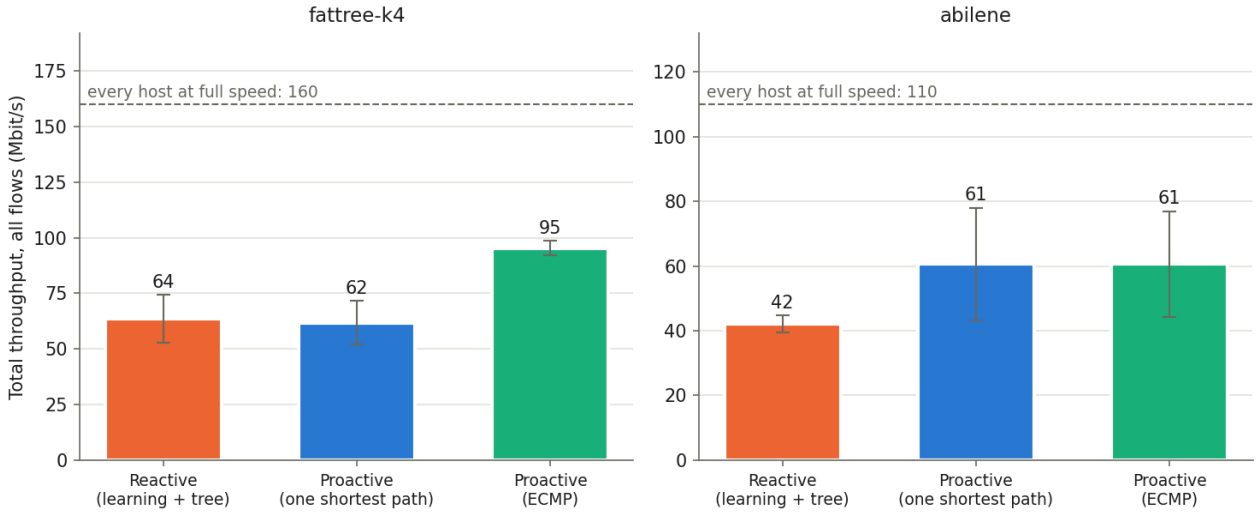
Network	Controller	Hop stretch mean / max	Delay stretch mean / max	Pairs on a longer path	Ping RTT (ms)
fat-tree	Reactive	**1.11 / 1.50**	-	**22%**	0.14
fat-tree	Proactive	1.00 / 1.00	-	0%	0.13
fat-tree	Proactive ECMP	1.00 / 1.00	-	0%	0.13
Abilene	Reactive	**1.40 / 3.00**	**1.58 / 4.47**	**70%**	**40.51**
Abilene	Proactive	1.02 / 1.33	1.00 / 1.00	0%	26.90
Abilene	Proactive ECMP	1.02 / 1.33	1.00 / 1.00	0%	26.24

Network	Controller	Total throughput, Mbit/s (ideal)	Jain index over links	Links unused
fat-tree	Reactive	63.6 ± 10.7 (160)	0.36	59%
fat-tree	Proactive	61.7 ± 9.8 (160)	0.33	63%
fat-tree	Proactive ECMP	**95.3 ± 3.3** (160)	**0.58**	**16%**
Abilene	Reactive	42.1 ± 2.6 (110)	0.56	29%
Abilene	Proactive	**60.7 ± 17.4** (110)	0.52	20%
Abilene	Proactive ECMP	**60.6 ± 16.3** (110)	0.52	20%

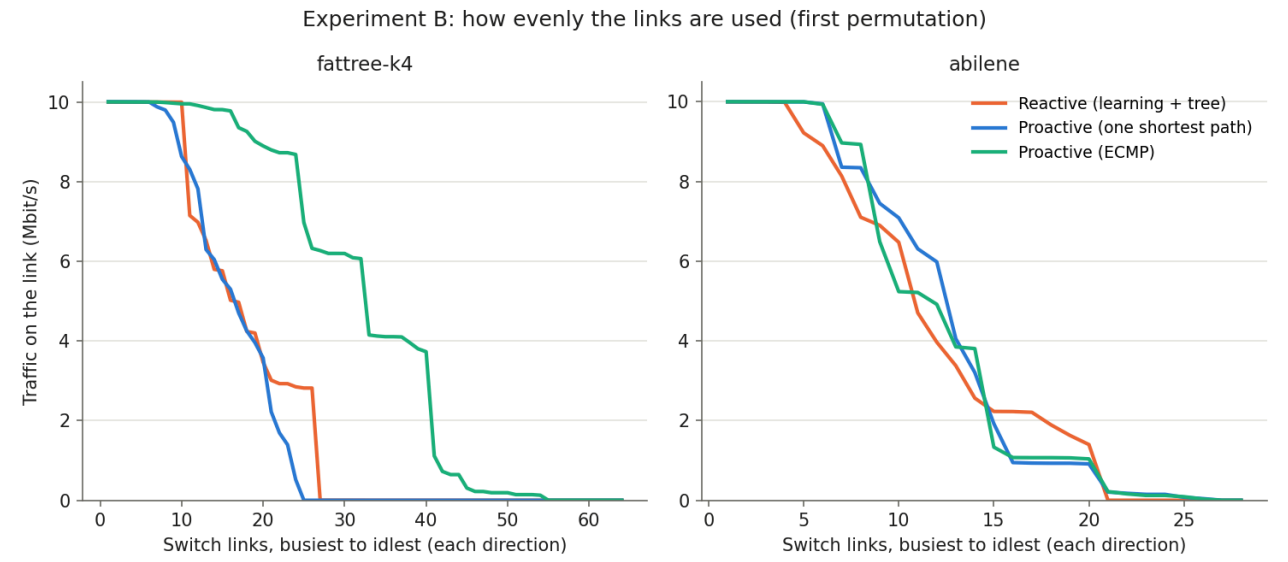
The busiest link was at 100% in every run, so the links (not the laptop) were the bottleneck. 0 packets were lost.



Path stretch



Throughput



Link balance

Findings

1. **ISP: reactive takes slow paths.**
    - 70% of pairs are on a slower path than necessary: +58% delay on average, up to 4.5× in the worst case.
    - Real pings are **40.5 ms against 26.9 ms**, and they match the map's prediction (39.3 vs 25.7 ms), which confirms the path tracing.
    - The cause is the spanning tree, which switches off 4 of Abilene's 14 links. Example: Atlanta → Washington has a direct 4.4 ms link, but reactive goes through Houston, Kansas City, Indianapolis, Chicago and New York (~23 ms).
2. **Data centre: one shortest path is not enough.**
    - One fixed shortest path gives no more throughput than reactive (62 vs 64 Mbit/s): both push traffic through the same core switch and leave ~60% of links idle.
    - **ECMP spreads the load: +50% throughput (95 Mbit/s), only 16% of links idle**, and the steadiest results.
3. **Nobody reaches the ideal.** ECMP on host pairs sometimes hashes two flows onto one link: the known ECMP collision problem (Al-Fares 2008, Hedera 2010).
4. **Reproducible.** A separate cloud-VM run produced exactly the same paths.

---

5. Experiment B: realistic traffic (university, data centre, ISP)

Why. Real traffic is mostly tiny flows, plus a few huge ones, arriving all the time. With reactive, every new flow costs a trip to the controller.

Scenarios. Each network carries the traffic measured in that kind of network:

Scenario	Network	Flow sizes	Flow arrivals	Source
University	campus tree, 16 hosts	87% < 10 KB, largest 10% carry 96% of bytes	Weibull gaps, 80% between 4 and 40 ms per edge switch (~200 flows/s)	Benson et al., IMC 2010 (Fig. 4a, Sec. 5, Table 4)
Data centre, web search	fat-tree k = 4	published table, mean 1.67 MB	Poisson, 30% load (~39 flows/s)	DCTCP (SIGCOMM 2010), as pFabric / PIAS `CDF_dctcp.tcl`
Data centre, data mining	fat-tree k = 4	published table: 80% < 10 KB, but 95% of bytes in flows > 35 MB	Poisson, 30% load (~57 flows/s)	VL2 (SIGCOMM 2009), as pFabric / PIAS `CDF_vl2.tcl`
ISP	Abilene: real link speeds (Atlanta–Indianapolis at ¼), city delays (netem)	SURGE web sizes: median 12 KB, mean 65 KB	**Real Abilene traffic matrices** for Mon 1 Mar 2004, 24 hours replayed in 30 s, scaled so each link is as busy as in 2004 (~57 flows/s)	Zhang et al., IMC 2005; SURGE (Barford & Crovella 1998)

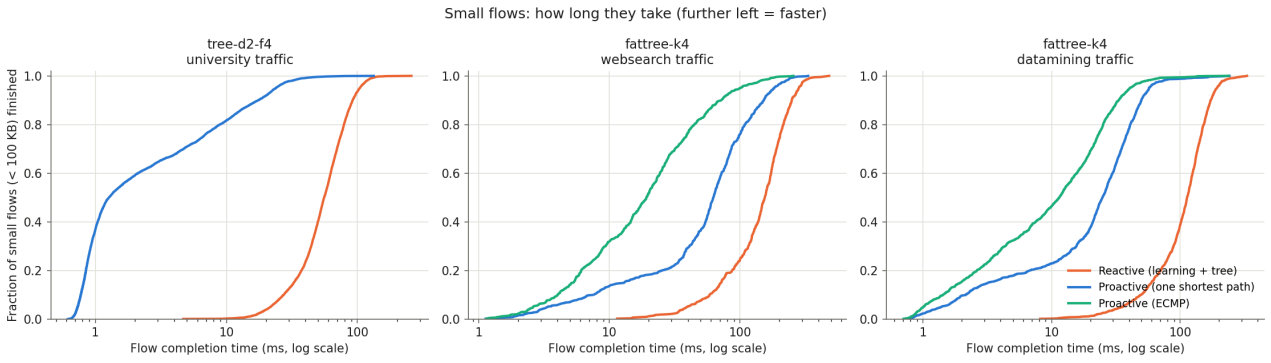
Measurement setup.

- **What a flow is:** a TCP connection that sends N bytes and waits for a 1-byte reply. Flow completion time (FCT) runs from connect to reply.
- **Links:** 100 Mbit/s. Flows over 20 MB are capped to 20 MB, which affects 0–4% of flows.
- **Same flows for everyone:** the flow schedule is computed in advance, so every controller gets exactly the same flows.

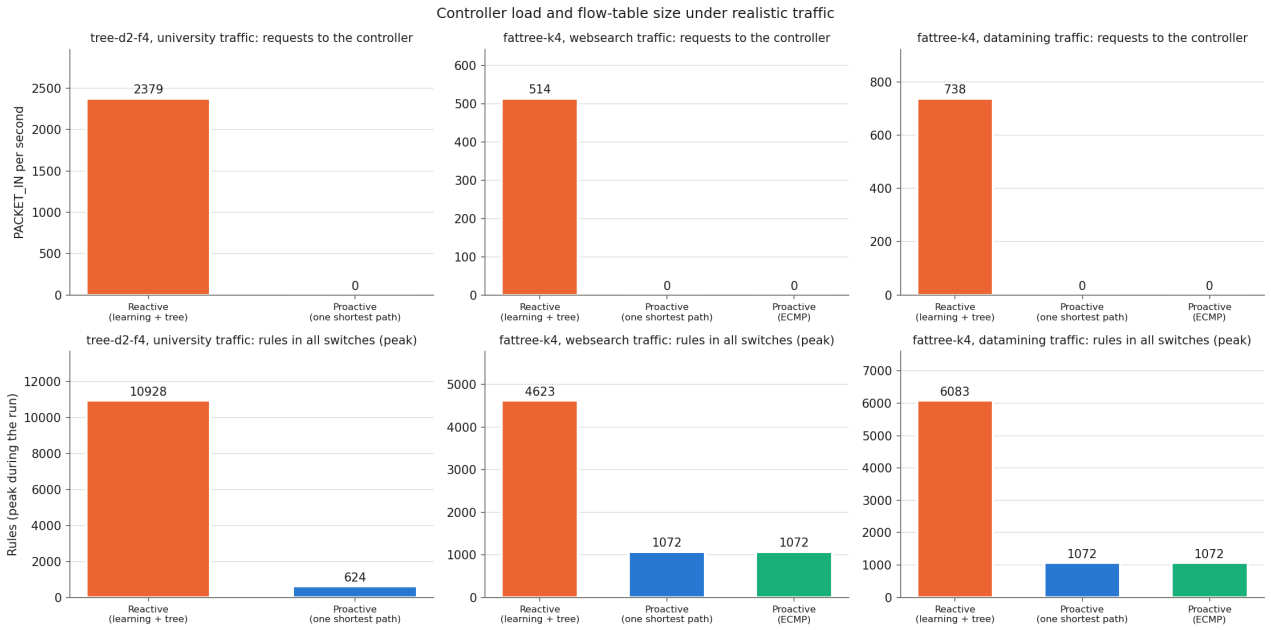
Results (laptop, 15 Oct; `results/measured/traffic-20261015-113417/` and `traffic-20261015-122618/`)

Scenario	Controller	Small flows < 100 KB, FCT median / p99 (ms)	Large flows > 10 MB, mean FCT (s)	PACKET_IN/s	Controller CPU	Peak rules: all switches / one switch
University	Reactive	**55.7** / 126.0	-	**2,379**	**86.4%**	**10,928 / 3,377**
University	Proactive	**1.3** / 37.9	-	0	0.1%	624 / 192
DC web search	Reactive	155.3 / 343.5	12.2	514	44.3%	4,623 / 779
DC web search	Proactive	62.6 / 242.0	12.8	0	0.3%	1,072 / 192
DC web search	Proactive ECMP	**19.2** / 170.8	**3.5**	0	0.3%	1,072 / **58**
DC data mining	Reactive	113.0 / 244.2	10.9	738	56.3%	6,083 / 1,003
DC data mining	Proactive	24.9 / 111.0	10.7	0	0.3%	1,072 / 192
DC data mining	Proactive ECMP	**11.5** / 67.7	**3.6**	0	0.3%	1,072 / **58**
ISP (Abilene)	Reactive	**120.0** / 405.7	-	624	44.0%	7,176 / 1,051
ISP (Abilene)	Proactive	**48.1** / 198.4	-	0	0.2%	386 / 62

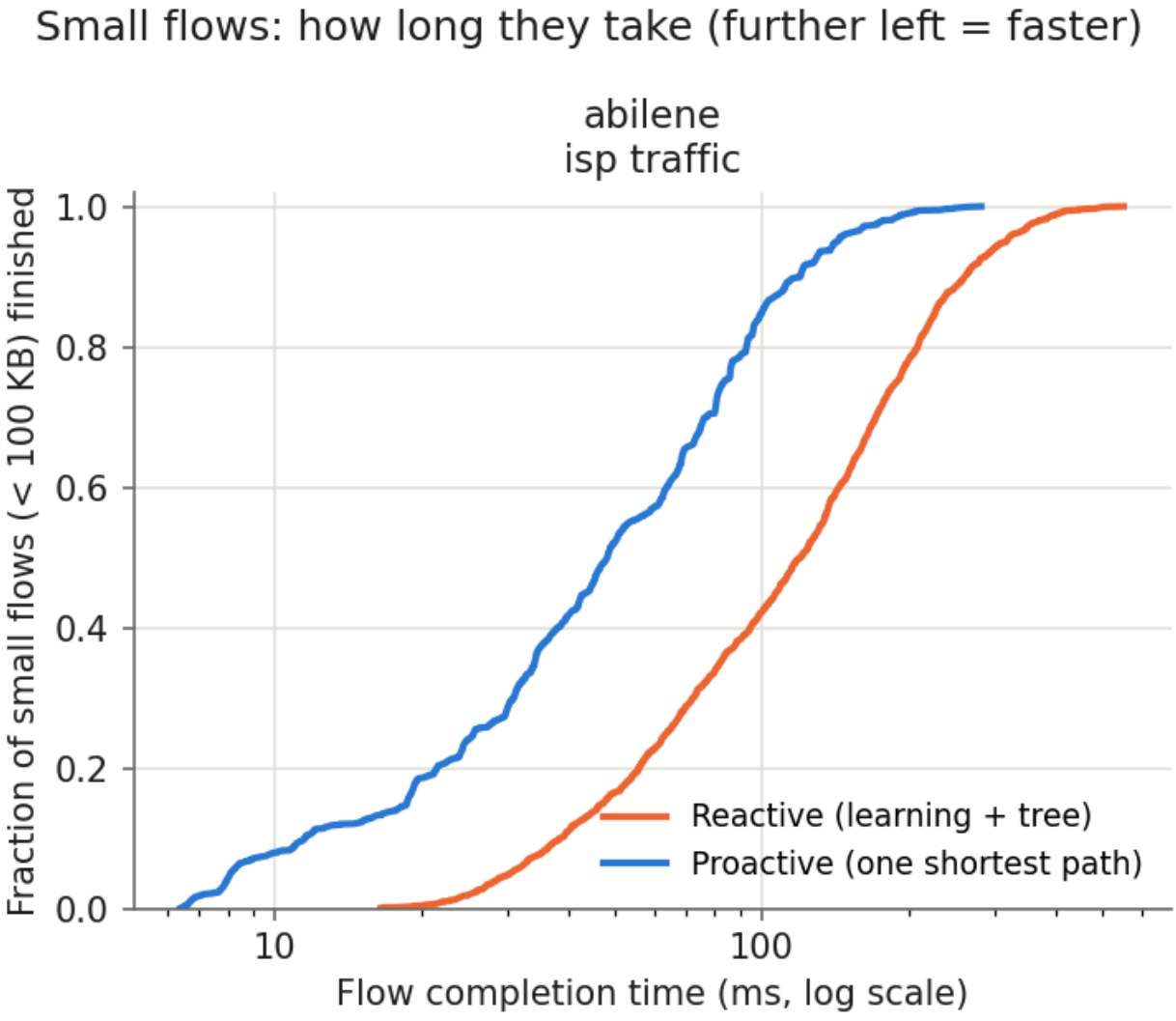
Flows completed: 6,102 university, 1,159 web search, 1,707 data mining and 1,761 ISP, for every controller, with **0 failures**.



Small-flow FCT, data-centre and university



Controller load and rules



Small-flow FCT, ISP

Findings

1. **University traffic: small flows are 44× slower with reactive** (median 55.7 vs 1.3 ms). The network is only 6% loaded, so this is purely the cost of asking the controller about every new flow.
2. **The controller nears saturation, and a busy controller causes even more work.**
   - **Load:** 203 new flows/s caused 2,379 PACKET_INs/s at 86% CPU (POX is single-threaded).
   - **More requests than needed:** each flow caused ~12 PACKET_INs, although it needs at most ~6. While the controller is slow, more packets of the same flow reach the switch before its rule does, and each one asks again.
   - **Lighter load, fewer requests:** a 10-s run at a similar rate had a less-loaded controller (54% CPU) and only 7.4 requests per flow. Requests per flow grow with controller load: a feedback loop towards overload.

3. **Reactive fills the switches:** 17× more rules on the campus tree (10,928 vs 624), and up to 3,377 in one switch. For scale, the HP 5406zl hardware switch studied in DevoFlow holds 16K exact-match rules, and our network has only 16 hosts.
4. **Data centre: the path matters as much as the controller.**
    - Small flows are **8–10× faster with proactive ECMP than with reactive** (web search 19 vs 155 ms; data mining 11.5 vs 113 ms).
    - Large flows depend on load balance, not on the controller: ~11–13 s with reactive or one shortest path, **3.5 s with ECMP**.
5. **ISP: reactive pays twice.**
    - Small flows are 2.5× slower (120 vs 48 ms). A small flow needs about two round trips.
    - **About 27 ms comes from reactive's longer paths:** 2 × 39.3 against 2 × 25.7 ms, using the round trips measured in Experiment A.
    - **About 40 ms comes from waiting for the controller** at each of the up-to-5 switches on the path.

---

6. Comparison with previous work

6.1 Our measurements next to published numbers

What	Published	Ours
Reactive first-packet delay	10 ms (NOX, in DIFANE); < 1.5 ms worst case (Ethane)	3.8–6.6 ms (laptop 1), 7.2–11.4 ms (laptop 2)
Proactive / data-plane first packet	0.4 ms (DIFANE)	0.67–1.5 ms (includes Open vSwitch's cache fill)
Controller capacity	NOX ≈ 30k flows/s, NOX-MT 1.6M/s (Tootoonchian 2012)	POX: 2,379 PACKET_IN/s at 86% CPU, near its limit
Flow sizes in campus / enterprise data centres	80% of flows < 10 KB (Benson 2010)	Our generator: 87% < 10 KB (uses Benson's curve)
ECMP leaves throughput on the table	Hash collisions limit ECMP (Al-Fares 2008, Hedera 2010)	ECMP reached 95 of 160 Mbit/s (60%), with the busiest links at 100%
Small rule budget serves most traffic	90% hits with < 5% of rules (CacheFlow 2016)	To test in the extension: our rule-budget sweep
Timeouts cut controller load	−58% cost (SmartTime 2014); −28% setups (AH-IHTA 2020)	To test: the long-timeout baseline in the extension

Numbers from different papers are not directly comparable (different hardware, controllers and years). **The ranges and trends agree.**

6.2 Realism: the paper behind every choice

`docs/Realism_Sources.md` lists the source of every network, traffic pattern and parameter: 16 references, each marked "checked in PDF", "checked online", "data file" or "to check".

- **Networks:** Al-Fares 2008 (fat-tree); Topology Zoo and the Abilene dataset's own topology file (Abilene).
- **Traffic:** Benson 2010 (university); DCTCP / VL2 through the exact pFabric/PIAS tables (data centre); the real Abilene traffic matrices of Zhang et al. 2005 (ISP).

6.3 Still to do: re-implementing previous methods

For the final submission we will run published ideas as baselines on our testbed, with the same traffic:

- **long / adaptive timeouts** (SmartTime, AH-IHTA);
- **immediate hybrid installation** (PARD-style);
- an **LSTM traffic predictor** (Batista 2024, which also used Mininet + POX).

---

7. What we learned so far

Situation	What decides performance	Winner
Many small flows, light load (university)	The cost of setting up each new flow	**Proactive**: 44× faster small flows; the controller idles
Heavy load (data centre)	Path choice and load balance	**Proactive ECMP**: 8–10× faster small flows, 3× faster large flows
Wide-area network (ISP)	Both: longer paths **and** controller set-up	**Proactive**: 2.5× faster small flows
Every pair active, no churn (Review I)	Nothing: same number of rules	Tie

Reactive loses on both counts. It pays a set-up cost for every new flow, and on networks with loops its paths are chosen by flooding, not by quality. Proactive's weak point is that it installs a rule for **every** pair, whether used or not; on larger networks this grows as N^2.

This is what our extension targets, the **idle-time hybrid**:

- pre-install rules only for the **popular** host pairs, on **balanced** paths, within a per-switch rule budget;

- do that work only when the controller is **idle**;
- handle the rest reactively.

The design and its literature gap are in `docs/literature/Literature_Review_IdleHybrid.md`.

---

8. Plan to the final submission (20 Nov)

When	Work
By 2 Nov	**Distributed setup (feedback point "multiple computers"):** POX on one PC, Mininet on another, to measure the effect of real control-channel delay. Then Mininet cluster mode across lab PCs.
3–10 Nov	**Idle-time hybrid controller** (`hybrid_eval.py`): popularity tracking (EWMA), busy/idle detection with hysteresis, budgeted pre-installation
10–15 Nov	**Baselines from previous work** (Section 6.3), all run on the same four realistic workloads
15–20 Nov	Final runs, repeated 5× with 95% confidence intervals; final report and demo

Risks.

- **POX near saturation:** POX is single-threaded and was at 86% CPU under university traffic, which also tells us where reactive breaks.
- **WSL2 timing noise:** repeats and a second laptop are used to control it.
- **One unverified parameter:** SURGE's numbers (ISP flow sizes) are still to be checked against the paper. The ISP traffic *volumes* are real data.

---

9. How to reproduce

```
git clone https://github.com/PrachitDeshinge-324/Openflow_SDN.git && cd Openflow_SDN
bash setup.sh                                                  # first time only (see INSTALL
sudo python3 experiments/run_experiments.py                    # Review I: latency, rules, ch
sudo python3 experiments/run_path_experiments.py               # Section 4: path quality (~15
sudo python3 experiments/run_traffic_experiments.py            # Section 5: realistic traffic
sudo python3 demo/run_demo.py                                  # live demo (reactive vs proac
```

Detailed documents:

- `docs/Path_Quality_Experiment.md`
- `docs/Realistic_Traffic_Experiment.md`

- `docs/Realism_Sources.md`
- `docs/literature/Literature_Review_IdleHybrid.md`
- `docs/Progress_Review_1.md`

Raw data: every run's raw data (each flow, each path, POX logs) is in `results/measured/`.